【扔物线 Compose 课程讲义】2-用 Compose 写出一个简单界面

声明式 UI

Compose 的「独立于平台」

Compose 的分包

  同一个组下多个分包

常用界面组件

Modifier

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当街课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：rengwuxian.com/cp

声明式 UI

- 关键在于「只需要声明」，而不是「界面看起来像声明」；
- 或者说「不需要命令式更新」。

Compose 的「独立于平台」

- 意思是「不依赖 Android」，好处是可以独立更新，以及加入多平台支持（包括更完美的预览功能）
- Compose 的「多平台」和 Flutter 的「跨平台」：关键不在于名字，而在于初始定位不同。Flutter 初始定位在跨平台，Compose 初始定位在推广 Kotlin

Compose 的分包

由下到上：

- compile：编译器插件部分
- runtime：最底层的概念模型
- ui：UI 相关的最基础支持
- animation：基于 runtime 和 ui 两层的动画 API 基础支持
- foundation：比 ui 层更实用的 UI 组件库
- material / material3：Material Design 2 / 3 的实现

同一个组下多个分包

一般引用和 group id 相同的 artifact 就够了，除了两个例外：

- ui 并不包含 ui-tooling 和 ui-tooling-preview
- material 包含 material-icons-core，但不包含 material-icons-extended

常用界面组件

- 最常见的写法：`ComponentActivity.setContent { }`
- `Text()`：文字
- `Image()`：图片
- `Icon()`：图标
- `Column()`、`Row()`：`LinearLayout`
- `Box()`：`FrameLayout`、`RelativeLayout`
- `ConstraintLayout()`：`ConstraintLayout`
- `RecyclerView`：`LazyColumn()` / `LazyRow()`

- `ScrollView`： `Modifier.vertical()`
- `ViewPager()`： `Pager()`
- `Button()`：Material 风格的按钮

Modifier

两个特点：

- 对顺序敏感；
- 多次调用会依次应用，而不是互相覆盖。

和函数参数的场景区别：

- 通用属性用 `Modifier`；
- 专用属性用函数参数。

几个课上讲到的 `Modifier`：

- `padding()`：增加边距；
- `background()`：设置背景；
- `clip()`：切边；
- `width()`、`height()`、`size()` 设置尺寸；
- `clickable()`：点击监听。

扔物线学堂 - 秉心(1437182)